Batch-Invariant LLM Serving on Hopper Without Custom Kernels

Isaac M. Thompson
Brigham Young University

Abstract

Greedy decoding in a batched inference server is not deterministic: the same request can produce different tokens depending on which other requests share its batch and whether its prefix is cached. vLLM's batch-invariant mode removes this dependence, but on an H100 it cuts decode throughput at batch 32 by 63.1% for a model without hand-tuned kernels. We show that on Hopper and Blackwell GPUs much of this cost can be avoided. vLLM routes Linear layers through a slower Triton kernel in this mode, a choice made after a model failed batch invariance with cuBLAS. Under the mode's settings, however, cuBLAS is batch-invariant for the large Linear shapes we tested: across 7 H100 builds, 2 B200 builds and 4 cuBLAS versions, 0 of 769,920 row comparisons differ under batch-invariant mode's flags. Routing Linear layers to cuBLAS gives identical tokens and logprobs in 6 of 7 serving configurations, including tensor parallelism, long context with chunked prefill and seeded sampling, while cutting the throughput loss to 0.2% for Qwen2.5-7B and 1.0% for Gemma-2-9B. The remaining configuration, Gemma-2-9B, is not batch-invariant under vLLM's own mode either; we localize that failure to FlashAttention's sliding-window path under CUDA graphs. At two-way tensor parallelism, the residual overhead comes from batch-invariant communication settings rather than the routing. Narrow layers such as mixture-of-experts routers can still fail with cuBLAS, so the routing has to be decided per shape.

1 Introduction

Greedy decoding is often assumed to be deterministic, but in a batched inference server it is not. The same request, served alone or alongside unrelated requests, with or without a cached prefix, can produce different tokens, because the floating-point reduction order of matrix multiplication and attention kernels depends on the shape of the batch [He and Thinking Machines Lab, 2025, Yuan et al., 2025]. In our measurements on vLLM [Kwon et al., 2023], 41 of 100 greedy math requests to Qwen2.5-7B-Instruct changed tokens across these conditions.

vLLM offers a batch-invariant mode that removes this dependence by replacing reduction kernels with fixed-order implementations. On an H100 it reduced decode throughput at batch 32 by 62.3% for Qwen2.5-7B-Instruct. vLLM has since added hand-tuned kernel configurations for a few models [vLLM contributors, 2026b], which recover most of the loss for those models only.

On Hopper (SM90) and Blackwell (SM100) GPUs, vLLM's batch-invariant mode sends every unquantized Linear layer through its Triton kernel, a choice made after GPT-OSS failed batch invariance with cuBLAS on H100 [vLLM contributors, 2026a]. We find that for the large Linear shapes that dominate inference cost, cuBLAS is batch-invariant under the mode's settings and even produces the same bits as the Triton kernel. Routing these layers to cuBLAS keeps outputs bitwise identical across serving conditions and removes nearly all of the cost. Narrow shapes, such as a mixture-of-experts router, are not covered by this finding. We contribute:

- a direct bitwise test showing that cuBLAS matrix multiplication is batch-invariant for the tested shapes under batch-invariant mode's flags on H100 and B200, across 4 cuBLAS versions (769,920 row comparisons, 0 mismatches);
- end-to-end serving tests over tensor parallelism, long context with chunked prefill, seeded sampling, and four model families, with preregistered pass/fail rules and controls;
- throughput measurements against vLLM's own batch-invariant mode on current vLLM main;
- a diagnosis of the two places it falls short: the TP=2 overhead comes from fixed-order communication, and a Gemma-2 failure lies in vLLM's FlashAttention path, not in the routing.

2 Background

A batch-invariant kernel computes each row of its output with the same sequence of floating-point operations regardless of how many other rows are in the batch. Reductions whose split depends on batch size (split-K matrix multiplication, split-KV attention, and some normalization kernels) break this property [He and Thinking Machines Lab, 2025]. vLLM's batch-invariant mode, enabled with `VLLM_BATCH_INVARIANT=1`, pins attention to a single split, replaces reductions with fixed-order kernels, disables reduced-precision reductions, prefers cuBLASLt, and pins collective communication to fixed algorithms. On Ampere (SM80) it overrides matrix multiplication with a persistent Triton kernel, since cuBLASLt there cannot be made batch-invariant. On SM90 and SM100 it instead sets cuBLAS workspace variables, with a source comment stating that split-K is the only source of batch variance on those GPUs. That comment is not accurate for every shape: after GPT-OSS failed batch invariance with cuBLAS on H100, `UnquantizedLinearMethod.apply` was changed to call the Triton `linear_batch_invariant` on every CUDA device [vLLM contributors, 2026a]. On Hopper the Linear layers, which perform most of the matrix-multiplication work, therefore run the Triton kernel.

3 Method

The change is one routing decision: under batch-invariant mode on SM90 and newer, Linear layers call `torch.nn.functional.linear` (cuBLAS) instead of the Triton kernel. Everything else in batch-invariant mode is unchanged. The routing applies to bf16 and fp16 inputs only: in the same bitwise test, cuBLAS fp32 matrix multiplication was not batch-invariant (rows differed at every batch size above one), while bf16 and fp16 with a bias term were. Our prototype applies this as an import hook that patches every vLLM process, including tensor-parallel workers, and logs a marker per process; a run counts only if the expected markers are present. An upstream version should also run a short bitwise self-test at startup and fall back to the Triton path if cuBLAS ever fails it, since NVIDIA does not document this behavior as a guarantee.

4 Evaluation

Setup. vLLM main (0.33.0rc1.dev454), PyTorch 2.14 with CUDA 13.0, bf16, NVIDIA H100 80GB unless noted. Determinism: each request is served 8 times in random order under four conditions (cold or warm prefix cache, alone or with 7 unrelated requests, each twice), and every pair of the 8 outputs is compared token by token; top-20 logprobs are also recorded. Prompts are

GPU, torch build	BI flags	Flags only	Defaults
B200, torch t2.13.0+cu130	0	0	19,544
B200, torch t2.14.0+cu130	0	0	19,544
H100, torch t2.11.0+cu128	0	0	7,344
H100, torch t2.12.0+cu126	0	0	6,784
H100, torch t2.13.0+cu126	0	0	6,784
H100, torch t2.13.0+cu129	0	0	7,344
H100, torch t2.13.0+cu130	0	0	8,544
H100, torch t2.14.0+cu126	0	0	6,784
H100, torch t2.14.0+cu130	0	0	8,544
H100, torch t2.14.0+cu130 + GPU load	0	0	8,544

Table 1: Rows that differ between a row multiplied alone and inside a batch, out of 38,496 per cell. BI flags: environment variables and PyTorch flags as batch-invariant mode sets them. Flags only: PyTorch flags without the cuBLAS environment variables. Defaults: PyTorch defaults (control).

few-shot GSM8K and MATH-500 problems. Pass/fail rules were written before each run (repository `PROTOCOL.md`); a control with batch-invariant mode off must diverge for a test to count.

Is cuBLAS batch-invariant? We multiply each row alone and inside batches of 1 to 8192 rows, at every Linear shape of six models, in bf16 and fp16, and compare rows near the start and end of the batch (Table 1). Under batch-invariant mode's flags, no row differed on any of 7 H100 builds or 2 B200 builds, including with a second process saturating the GPU. For these shapes, setting only the PyTorch flags, without the cuBLAS workspace environment variables, made no difference; for narrower shapes it does, as described below. With PyTorch defaults, between 6,784 and 8,544 of 38,496 rows differed on H100, and 19,544 on B200, so the test detects batch dependence when it exists.

Which shapes? A second, denser test checked every batch size from 1 to 1024 and every sixteenth up to 8192, on 37 shapes including mixture-of-experts routers and a sweep of narrow output widths N. On H100, at $K = 2880$ and 4096, every shape with $N \leq 96$ is batch-variant under the same settings, and every shape with $N \geq 128$ is batch-invariant at every batch size, in both bf16 and fp16. A finer sweep shows that the boundary also depends on K and on the cuBLAS workspace environment variables, which vLLM sets at runtime: without them, widths of 128 and 256 at $K \geq 2880$ also fail. With them, some narrow widths are invariant at other K values, and widths between 97 and 127 are invariant only when they are multiples of 8. Every width of at least 128 that we tested was invariant. The gpt-oss-20b router ($N = 32$, $K = 2880$) diverges from batch size 23 in bf16 and 2 in fp16, matching another contributor's report on H200. Routing therefore has to be per shape: wide projections, which carry nearly all the cost, can use cuBLAS, while narrow layers such as routers must keep the Triton kernel. Because the property is observed rather than documented, the upstream design we propose checks each Linear shape bitwise at model load and routes only shapes that pass. On Blackwell, cuBLAS 13.4 documents an experimental batch-invariance mode for compute capability 10.x, which is the natural long-term path there; it does not cover Hopper.

End-to-end determinism. With Linear layers on cuBLAS, 6 of 7 serving configurations produced identical tokens and identical top-k logprobs across all conditions (Table 2). Controls with batch-invariant mode off diverged on 4% to 76% of requests. Gemma-2-9B fails: 39 of 100 requests diverged with cuBLAS routing, and 40 with vLLM's stock batch-invariant mode, against 60 with

Configuration	Plain cuBLAS	Mode off	Markers
Qwen2.5-7B, TP=2	0 / 100	38 / 100	2
Qwen2.5-7B, TP=4 (TP=2 control)	0 / 100	38 / 100	4
Qwen2.5-7B, long context, chunked prefill	0 / 50	29 / 50	1
Qwen2.5-7B, seeded sampling	0 / 100	40 / 100	1
Qwen2.5-32B, TP=2	0 / 100	76 / 100	2
Mistral-7B	0 / 100	4 / 100	1
Gemma-2-9B	39 / 100	60 / 100	1

Table 2: Requests whose 8 runs were not all identical, with Linear layers on cuBLAS under batch-invariant mode (plain cuBLAS) and with batch-invariant mode off (control). Markers count processes that loaded the routing change.

Model	Stock	Plain cuBLAS	Plain, worst repeat
Qwen2.5-7B	63.1%	0.2%	3.1%
Gemma-2-9B	59.2%	1.0%	1.3%
Mistral-7B	5.8%	3.3%	5.9%
Qwen2.5-7B, TP=2	61.0%	5.8%	6.8%
Qwen2.5-32B, TP=2	58.2%	−2.3%	−1.0%

Table 3: Decode throughput loss at batch 32 against batch-invariant mode off. Median of three jobs.

the mode off. vLLM's batch-invariant mode does not cover Gemma-2 in this setup; the routing change neither causes nor fixes this. Varying one component at a time under stock batch-invariant mode (30 requests each) localizes the failure: switching attention to the Triton backend removes it (0 diverging requests, against 19 for its control), as does eager execution (0, against 23). Removing logit softcapping does not (15, against 24), and Gemma-7B, which shares Gemma-2's head size but has no softcapping, is unaffected (0, against 30). Two further variants narrow it: disabling Gemma-2's sliding-window attention removes the divergence (0, against 17), and so does turning off CUDA graphs while keeping compilation (0, against 15). The failure therefore lies in FlashAttention's sliding-window path under CUDA graphs.

Throughput. Table 3 reports decode throughput loss at batch 32 against batch-invariant mode off, the median of three independent jobs. For models without tuned configurations, stock batch-invariant mode loses 63.1% (Qwen2.5-7B) and 59.2% (Gemma-2-9B); cuBLAS routing loses 0.2% and 1.0%. Mistral-7B shares most Linear shapes with Llama-3.1-8B, which vLLM has tuned, so stock mode is already cheap there (5.8%). For prefill, cuBLAS routing loses at most 5.0% against 9.5% to 25.5% for stock mode. 16 of 17 measured cells meet our preregistered limit of 5%. The exception is Qwen2.5-7B at TP=2, which loses 5.8% (stock: 61.0%). This excess comes from batch-invariant mode's fixed-order communication settings, not from the routing change: applying only those settings, with batch-invariant mode otherwise off, costs 6.0% on its own. Cheaper deterministic communication is a separate target.

5 Limitations

The bitwise property is observed, not guaranteed by NVIDIA, and it does not hold for every shape: another contributor found a mixture-of-experts router shape (N=32, K=2880) that is not batch-

invariant under the same settings, so an upstream change must decide per shape, for example with a bitwise check of each Linear shape at model load. We tested one engine, single-node tensor parallelism up to four GPUs, dense models up to 32B parameters, and bf16; we did not run serving tests on H200, multi-node serving, mixture-of-experts models, quantized models, or production traffic. Gemma-2-9B is not batch-invariant under vLLM's batch-invariant mode with its default FlashAttention backend. For models in vLLM's tuned table the benefit is small. We compare only against vLLM's own batch-invariant mode, not against other engines' deterministic modes or LLM-42 [Gond et al., 2026].

6 Related Work

He and Thinking Machines Lab [2025] identify batch-size-dependent reductions as the main source of inference nondeterminism and describe batch-invariant kernels; vLLM's batch-invariant mode follows this design. Yuan et al. [2025] show that numerical nondeterminism changes reasoning accuracy across batch sizes, GPUs, and precisions. LLM-42 [Gond et al., 2026] obtains determinism through verified speculation rather than invariant kernels. Our contribution is narrower: on current NVIDIA hardware, under the right flags, the vendor matrix-multiplication library is batch-invariant for the large Linear shapes that dominate cost, so the custom kernel is unnecessary for those shapes. It remains necessary for others, such as small mixture-of-experts routers.

7 Conclusion

Reproducible serving has been treated as a trade between determinism and throughput. On current NVIDIA hardware, that trade is mostly unnecessary for dense Linear layers: the vendor library already provides batch invariance under the right flags, and routing to it recovers nearly all of the lost throughput. We hope this makes batch-invariant serving a reasonable default for evaluation and for reinforcement-learning rollouts, where reproducibility matters most. An upstream version of the change should keep a startup self-test, since the property is observed rather than guaranteed. Code, raw outputs, and the decision rules for every experiment are at `https://github.com/imthomp/vllm-batch-invariance`.

References

Raja Gond, Aditya K. Kamath, Ramachandran Ramjee, and Ashish Panwar. LLM-42: Enabling determinism in LLM inference with verified speculation, 2026.

Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Thinking Machines Lab: Connectionism, September 2025. URL `https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/`.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

vLLM contributors. [bugfix][model] fix gpt-oss batch invariance. vLLM pull request 35404, merged February 2026, 2026a. URL `https://github.com/vllm-project/vllm/pull/35404`.

vLLM contributors. [kernel][perf] add H100 tuned configs for Llama-3.1-8B and Qwen3-8B to the batch-invariant matmul table. vLLM pull request 62219, 2026b. URL https://github.com/vllm-project/vllm/pull/62219.

Jiayi Yuan, Hao Li, Xinheng Ding, Wenya Xie, Yu-Jhe Li, Wentian Zhao, Kun Wan, Jing Shi, Xia Hu, and Zirui Liu. Understanding and mitigating numerical sources of nondeterminism in LLM inference, 2025.